Asteroids – LUA & LOVE2D Game Development Project

By George Nicholl

Overview

This project is a minimalist clone of the classic Asteroids Arcade game. It was developed in the language LUA and using the LOVE2D framework. The game demonstrates fundamental principles of game development including state management, collision detection and modular programming.

Figure 1: Main Menu of Asteroids Game

Technology Stack

Component	Description
Programming Language	LUA
Framework	LOVE2D (v11.x)
Paradigms	Modular Design, state machines, object-oriented structure
IDE / Tools Used	Visual Studio Code, LOVE2D Debug Console

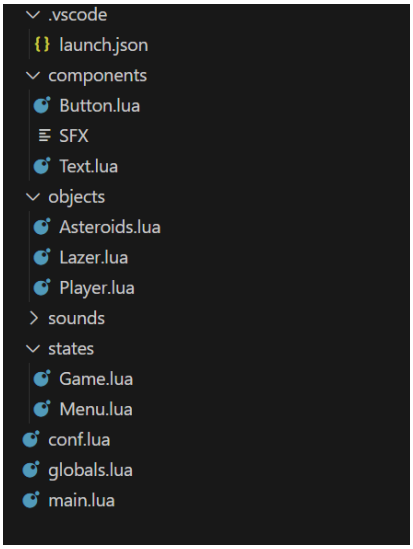

Figure 2: Project Folder Structure

At a Glance:

- Player Spaceship with thrust, rotation and shooting mechanics
- Randomly generated asteroids with wrap-around movement
- Collison Detection and destruction mechanics
- Menu & game state transitions
- Modular code organisation for scalability

Project Goals & Learning Outcomes

Project Goals

The aim of this project was to recreate the classic Asteroids arcade game while strengthening my understanding of game development fundamentals. I focussed on building clean, modular code and implementing mechanics that reflect real game design challenges such as state transitions and responsive controls.

Learning Outcomes

Through this project I developed and refined skills in:

- Game State Management – separating menu, gameplay, pause and end states.
- Collision Detection – Applying circle-based distance checks for lasers, player and asteroids.
- Object-Oriented Programming in LUA – structuring reusable components (Players, Asteroids, Laser).
- Event Handling – mapping multiple input methods (keyboard & mouse) to responsive gameplay,

- Modular Architecture – organising components, objects and states to ensure readability and scalability.

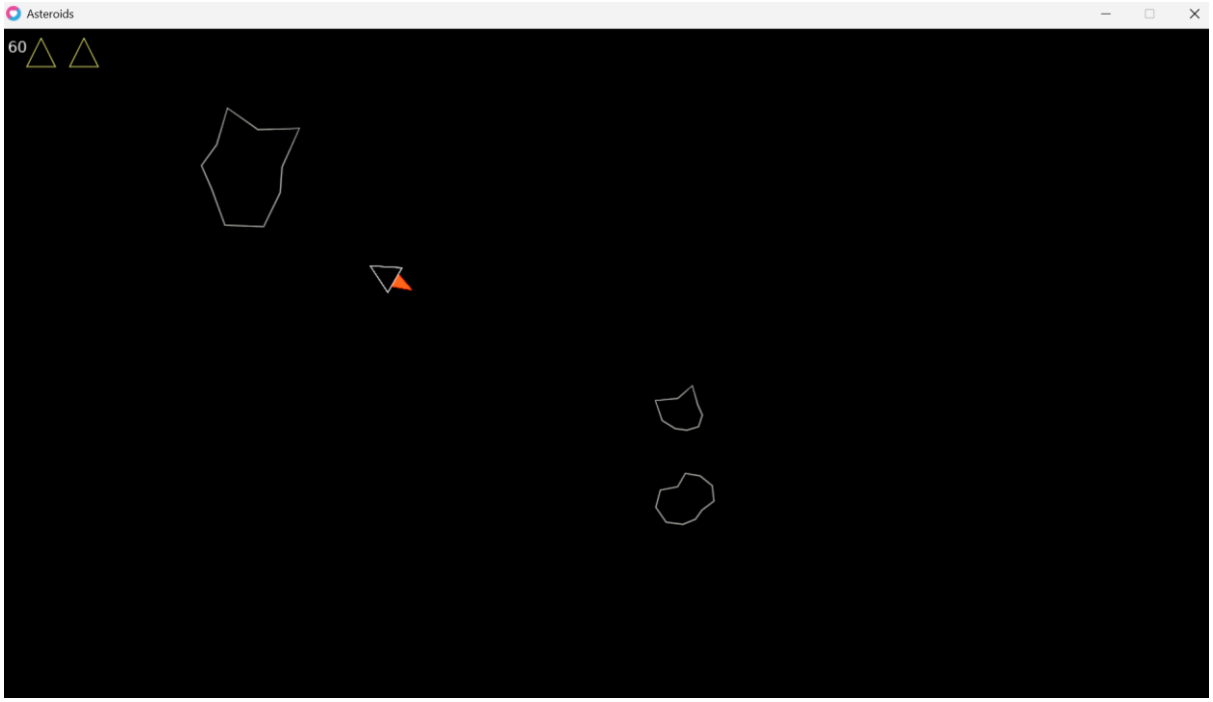

Figure 3: Gameplay screenshot shwoing player's spaceship & surrounding asteroids during active play

System Design & Architecture

This project follows a modular architecture to separate responsibilities and make the codebase scalable and easy to maintain.

High Level Architecture

Main Loop (main.lua)

- Entry point of the game. Handles initialisation, update, and draw functions from LOVE.

Global Configuration (conf.lua, globals.lua)

- States window setup, constants, and shared helper function.

States (states/)

- Implements a finite state machine: Menu – Running – Paused – Ended.

Objects (objects/)

- Encapsulates game entities (Player, Asteroids, Lasers).

Components (components/)

- Reusable UI elements such as buttons and text,

Data Flow

1. User Input – Captured in love.keypressed, love.keyreleased, and love.mousepressed.
2. State Manager – Directs Input and updates to the active state (Menu/Game).
3. Game Objects – Player, Lasers, and asteroids update their positions and detect collisions.
4. Rendering – The active states draw() method calls object render functions and UI components.

Architectural Design

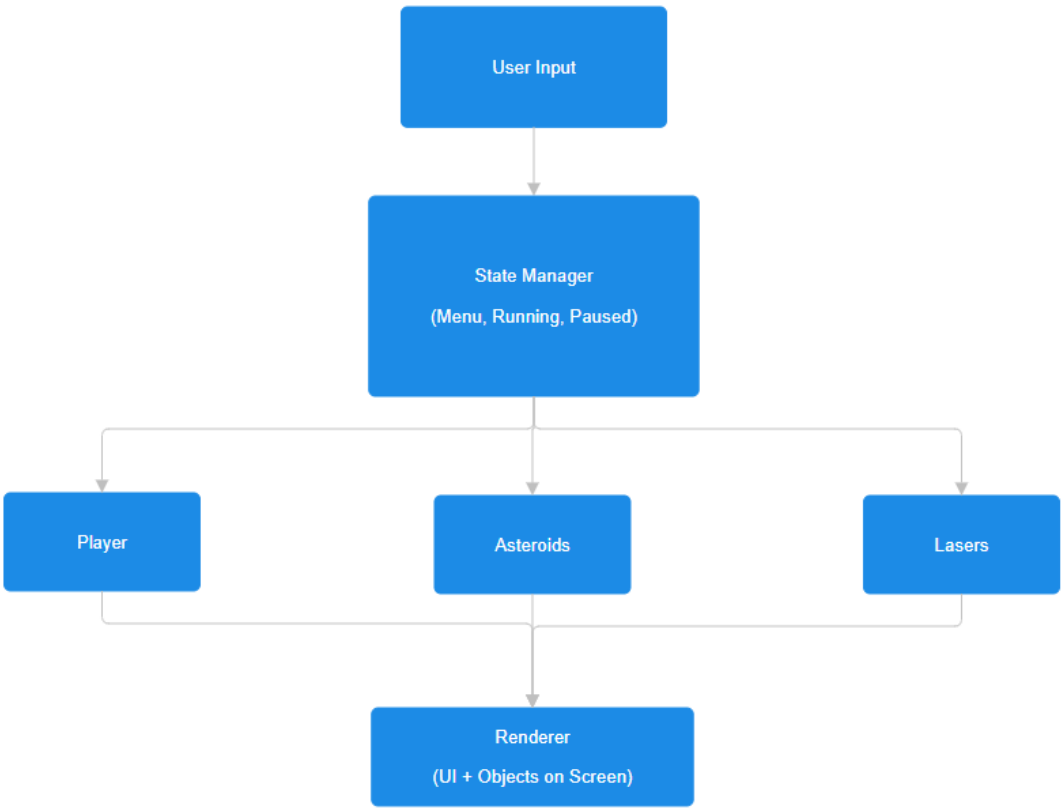

Figure 4: Data flow diagram illustrating how user input passes through the state manager and updates game objects

Gameplay Mechanics & Features

Core Gameplay

- The player controls a triangular spaceship in a wrap-around area.
- The objective is to survive and destroy as many asteroids as possible.
- The game ends when the player loses all lives.

Player Controls

- Rotation – A/D or Arrow Keys to turn left/right.
- Thrust – W or Up Arrow to accelerate forward.
- Shoot – Space to fire lasers from the ship's nose.
- Pause – Esc to toggle between running and paused states.

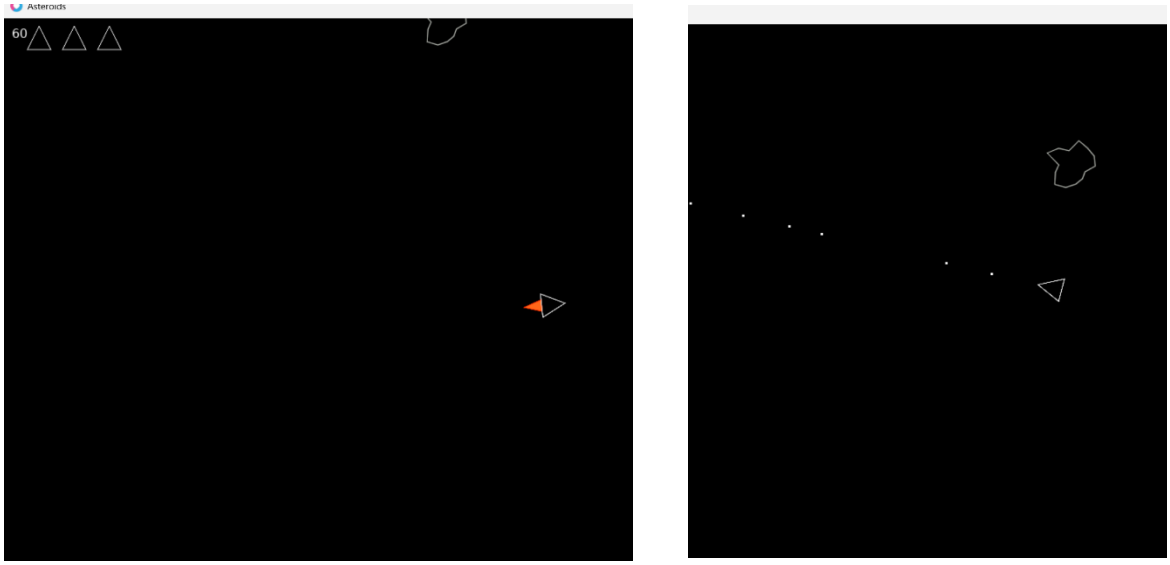

Figure 5: Screenshots showing both thrust and shooting gameplay

Asteroids

- Spawn randomly at the start of each leel.
- Travel in random directions with wrap-around movement. (disappearing on one edge, reappearing on the opposite).
- Break into smaller pieces.

Figure 6: Screenshot showing random asteroid spawning

Lasers

- Fired by the player in the direction the ship is facing
- Destroy asteroids on collision.
- Limited in number to prevent spamming.

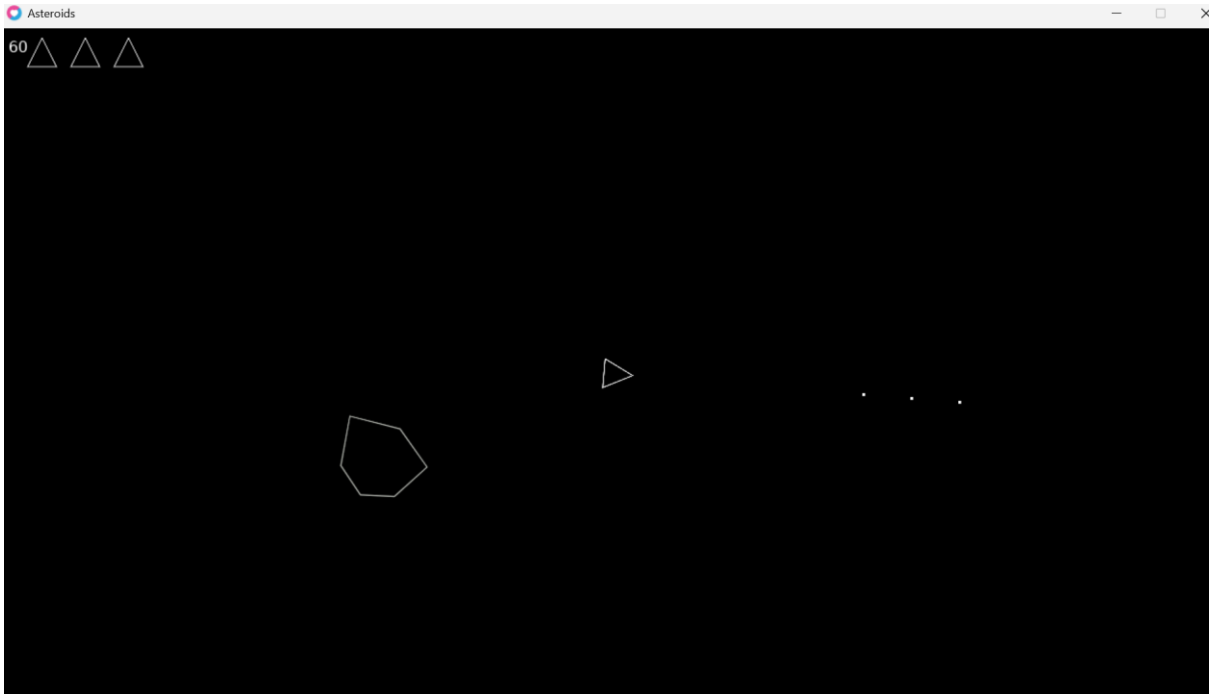

Figure 7: Screenshot shwoing laser shooting

Collision Detection

- Circle-Based distance calculation between objects (player-asteroid, laser-aasteroid)
- On collision:
- - Laser + Asteroid – Asteroid is Destroyed
- - Player + Asteroid – player explode, loses one life, and respawns after a delay.

Game States

- Menu – Player can start a new game
- Running – Active Gameplay
- Paused – Temporarily stops updates
- Ended – Shown when all lives are lost.

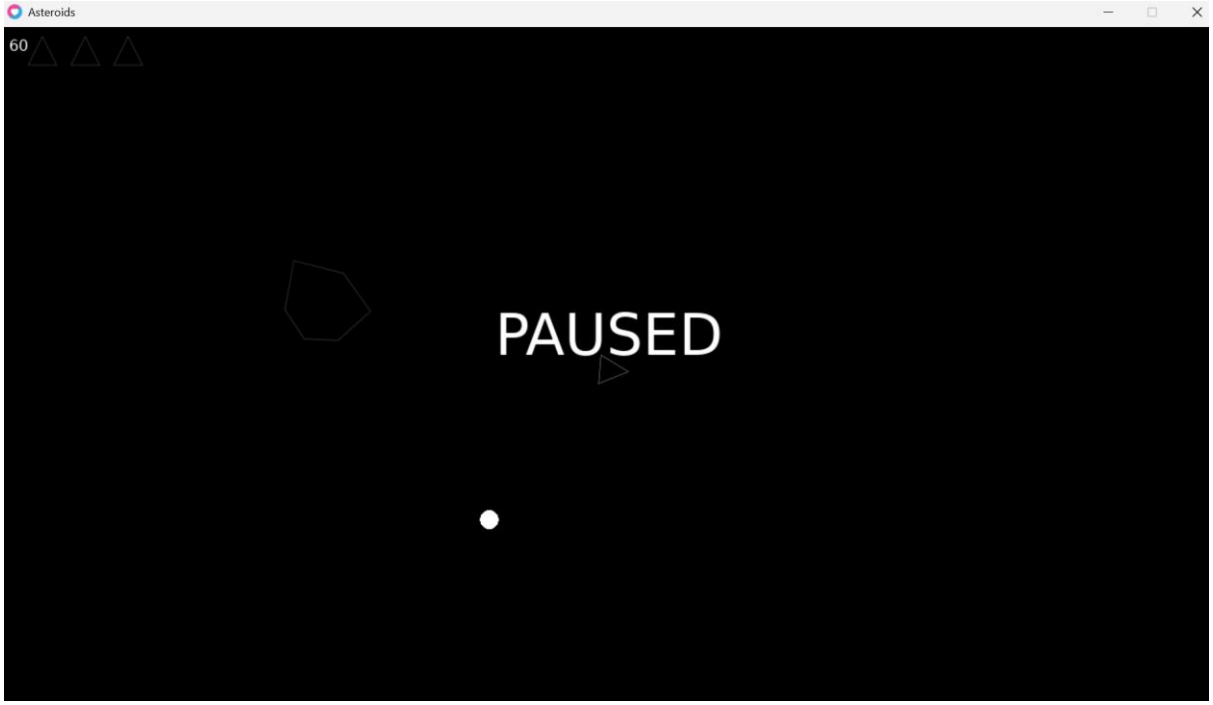

Figure 8: Screenshot showing paused game state

Lives System

- Player begins with 3 lives.
- Each collision reduces lives by 1.
- Explosions and respawn animations visually represent the loss.

Figure 9: Screenshot showing lives

Testing & Debugging

Test Case	Expected Result	Outcome
Player shoots asteroid	Asteroid destroyed, score increased	Successful ☑
Player Collides with asteroid	Lose one life, respawn triggered	Successful☑
Laser expires after delay	Laser disappears off screen	Successful☑
Esc key pressed	Game Paused	Successful☑
Asteroids wrap around screen	Reappear on opposite edge	Successful☑

Debugging Techniques:

- Used print() statements to track object positions and collisions.
- Implemented temporary hitboxes to visualise object bounds.
- Tested each state independently before integration.

Challenges and Solutions

Challenge	Solution / Approach
Managing multiple game states (menu, running, paused, ended)	Implemented a finite state machine using separate LUA files for each state.
Collision precision and performance	Used circle-based distance checks instead of bounding boxes for efficient detection.
Object wrapping and repositioning	Applied modular arithmetic to reappear objects on opposite screen edges.
Code Scalability	Designed a modular architecture separating objects, states, and UI components.
Maintaining consistent FPS	Optimised object updates and draw calls to avoid redundant loops.

Figure 10: Gameplay scene during Level 4 demonstrating multiple active asteroids and smooth performance at 60 FPS

Future Improvements

In future versions, the following features could enhance gameplay and user experience:

- Score Multiplier or combo system for consecutive hits
- Background music and explosion sound effects
- Local high score saving via the LOVE2D filesystem API
- Mobile compatibility with on-screen touch controls
- Polished UI transitions with particle effects.

Conclusion

The Asteroids project successfully replicates the core mechanics of the original arcade classic while demonstrating modern programming practices in LUA and the LOVE2D framework.

Through iterative development, modular design and object-oriented architecture, it delivers smooth gameplay and robust collision detection.

This project strengthened my understanding of game loops, input handling and state management, providing a solid foundation for future game and software development work.